PROJECT: ANALYZE YOUR STOCK PORTFOLIO FOR RISKS AND RETURNS

datalab

Asset allocation is the most important decision that any investor needs to face. They need to decide how to spread their total capital over certain assets (in this case, stocks). When considering the allocation, the investor wants to balance the risk and the potential reward. At the same time, the allocation depends on factors such as individual goals, risk tolerance, and the investment horizon.

The key framework used in asset allocation is the Modern Portfolio Theory (MPT), which was introduced by the Nobel Prize winner Harry Markowitz. MPT describes how investors can construct portfolios to maximize their expected returns for a given level of risk or, conversely, minimize risk for a given level of expected return. The mathematical framework used to achieve this is called mean-variance optimization.

The main insight from MPT is that investors should not evaluate an asset's performance alone. Instead, they should evaluate how it would impact the performance of a portfolio of assets. Another important takeaway is the concept of diversification, which means that owning different kinds of assets reduces risk. That is because the loss or gain of a particular security has less impact on the overall portfolio's performance.

Your task

In the dynamic realm of finance, data scientists/analysts are often tasked with finding optimal investment strategies. Imagine you're one such analyst, and you were asked to build an effective portfolio comprising FAANG stocks – Facebook (Meta), Apple, Amazon, Netflix, and Google. Your goal is to maximize returns while mitigating risk.

In this project, you are tasked to find the optimal allocation to the FAANG stocks based on historical stock price data spanning the years 2020-2023. The dataset is stored in the `faang_stocks.csv` file. For each trading day, it contains the close prices of the five tech companies.

```python
# Importing libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from pypfopt.efficient_frontier import EfficientFrontier
from pypfopt import risk_models
from pypfopt import expected_returns

# Setting the plotting style to be colorblind-friendly
plt.style.use("seaborn-colorblind")

# Loading data
stock_prices_df = pd.read_csv("faang_stocks.csv", index_col="Date")

# Changing the index to a datetime type allows for easier filtering and plotting.
stock_prices_df.index = pd.to_datetime(stock_prices_df.index)
stock_prices_df
```

Date	AAPL	AMZN	GOOGL
2020-01-02T00:00:00.000	75.09	94.9	
2020-01-03T00:00:00.000	74.36	93.75	
2020-01-06T00:00:00.000	74.95	95.14	
2020-01-07T00:00:00.000	74.6	95.34	
2020-01-08T00:00:00.000	75.8	94.6	
2020-01-09T00:00:00.000	77.41	95.05	
2020-01-10T00:00:00.000	77.58	94.16	
2020-01-13T00:00:00.000	79.24	94.57	
2020-01-14T00:00:00.000	78.17	93.47	
2020-01-15T00:00:00.000	77.83	93.1	
2020-01-16T00:00:00.000	78.81	93.9	
2020-01-17T00:00:00.000	79.68	93.24	
2020-01-21T00:00:00.000	79.14	94.6	
2020-01-22T00:00:00.000	79.43	94.37	
2020-01-23T00:00:00.000	79.81	94.23	
2020-01-24T00:00:00.000	79.58	93.08	

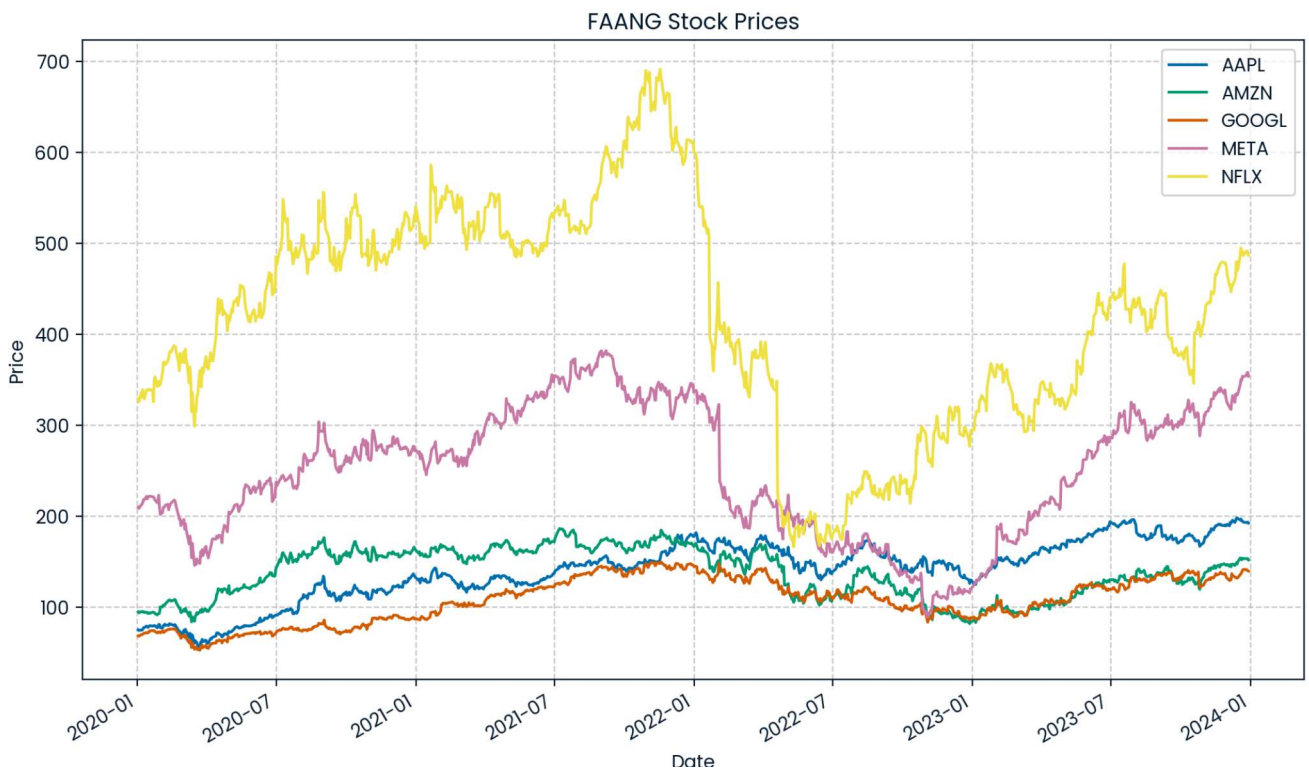
Rows: 1,006 Expand

```python
# ==========================================
# Portfolio Optimization for FAANG Stocks
# ==========================================
"""
Uses Modern Portfolio Theory to find:
1. Equal-Weight (1/n) Benchmark
2. Minimum Volatility Portfolio (PyPortfolioOpt & scipy.optimize)
3. Maximum Sharpe Ratio Portfolio
"""

# ==========================================
# 1. Imports
# ==========================================
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import scipy.optimize as sco
from pypfopt.efficient_frontier import EfficientFrontier
from pypfopt import risk_models, expected_returns

# ==========================================
# 2. Data Loading & Preprocessing
# ==========================================
# Ensure 'faang_stocks.csv' is in the working directory
stock_prices_df = pd.read_csv("faang_stocks.csv", index_col="Date")
stock_prices_df.index = pd.to_datetime(stock_prices_df.index)

# Calculate daily returns
returns_df = stock_prices_df.pct_change().dropna()
n_assets = len(returns_df.columns)

print("✅ Data loaded successfully. Shape:", stock_prices_df.shape)
print(f"📊 Analyzing {n_assets} assets: {', '.join(returns_df.columns)}\n")

# Optional: Plot historical prices
plt.style.use("seaborn-colorblind")
stock_prices_df.plot(title="FAANG Stock Prices", figsize=(10, 6))
plt.ylabel("Price")
plt.grid(True, linestyle="--", alpha=0.6)
plt.tight_layout()
plt.show()

# ==========================================
# 3. Task 1: Equal-Weight (1/n) Benchmark
# ==========================================
weights_1n = np.array([1 / n_assets] * n_assets)
portfolio_returns = returns_df.dot(weights_1n)

benchmark_exp_return = portfolio_returns.mean() * 252
benchmark_vol = portfolio_returns.std() * np.sqrt(252)
benchmark_sharpe = benchmark_exp_return / benchmark_vol if benchmark_vol != 0 else 0.0

print("=" * 60)
print("TASK 1: EQUAL-WEIGHT (1/N) PORTFOLIO")
print("=" * 60)
print(f"Expected Annual Return: {benchmark_exp_return:.4f}")
print(f"Annual Volatility:      {benchmark_vol:.4f}")
print(f"Sharpe Ratio:           {benchmark_sharpe:.4f}")

# ==========================================
# 4. Task 2: Minimum Volatility Portfolio
# ==========================================
# Prepare annualized inputs for optimization
avg_returns = returns_df.mean() * 252
cov_mat = returns_df.cov() * 252

# Approach A: PyPortfolioOpt
ef_mv = EfficientFrontier(avg_returns, cov_mat)
weights_mv_pypfopt = ef_mv.min_volatility()
mv_pypfopt = pd.Series(weights_mv_pypfopt, index=avg_returns.index)
mv_exp_ret, mv_vol, mv_sharpe = ef_mv.portfolio_performance(risk_free_rate=0)
```

```python
print("\n" + "=" * 60)
print("TASK 2: MINIMUM VOLATILITY PORTFOLIO")
print("=" * 60)
print("[Approach A: PyPortfolioOpt]")
print("Optimal Weights:")
print(mv_pypfopt.round(4))
print(f"Expected Annual Return: {mv_exp_ret:.4f}")
print(f"Annual Volatility:      {mv_vol:.4f}")
print(f"Sharpe Ratio:           {mv_sharpe:.4f}")

# Approach B: Manual optimization using scipy (Educational)
def portfolio_volatility(weights, cov_mat):
    return np.sqrt(np.dot(weights.T, np.dot(cov_mat, weights)))

bounds = tuple((0, 1) for _ in range(n_assets))
initial_guess = [1 / n_assets] * n_assets
constraint = {"type": "eq", "fun": lambda x: np.sum(x) - 1}

result = sco.minimize(
    portfolio_volatility,
    x0=initial_guess,
    args=(cov_mat,),  # scipy expects args as a tuple
    method="SLSQP",
    constraints=constraint,
    bounds=bounds,
)

mv_scipy = pd.Series(result.x, index=avg_returns.index).round(4)
mv_scipy_vol = result.fun

print("\n[Approach B: scipy.optimize (Manual)]")
print("Optimal Weights:")
print(mv_scipy)
print(f"Annual Volatility:      {mv_scipy_vol:.4f}")
print(f"Optimization Success:   {result.success}")

# ==========================================
# 5. Task 3: Maximum Sharpe Ratio Portfolio
# ==========================================
# Use PyPortfolioOpt's built-in robust estimators (returns annualized values by default)
avg_returns_alt = expected_returns.mean_historical_return(stock_prices_df, compounding=False)
cov_mat_alt = risk_models.sample_cov(stock_prices_df)

ef_ms = EfficientFrontier(avg_returns_alt, cov_mat_alt)
weights_ms = ef_ms.max_sharpe(risk_free_rate=0)
ms_portfolio = pd.Series(weights_ms, index=avg_returns_alt.index)
ms_exp_ret, ms_vol, ms_sharpe = ef_ms.portfolio_performance(risk_free_rate=0)

print("\n" + "=" * 60)
print("TASK 3: MAXIMUM SHARPE RATIO PORTFOLIO")
print("=" * 60)
print("Optimal Weights:")
print(ms_portfolio.round(4))
print(f"Expected Annual Return: {ms_exp_ret:.4f}")
print(f"Annual Volatility:      {ms_vol:.4f}")
print(f"Sharpe Ratio:           {ms_sharpe:.4f}")

# ==========================================
# 6. Summary Comparison Table
# ==========================================
summary_df = pd.DataFrame({
    "Portfolio": [
        "1/n Equal-Weight",
        "Min Volatility (PyPortfolioOpt)",
        "Min Volatility (scipy)",
        "Max Sharpe Ratio"
    ],
    "Exp. Return": [benchmark_exp_return, mv_exp_ret, np.nan, ms_exp_ret],
    "Volatility":  [benchmark_vol, mv_vol, mv_scipy_vol, ms_vol],
    "Sharpe Ratio":[benchmark_sharpe, mv_sharpe, np.nan, ms_sharpe]
})

print("\n" + "=" * 60)
```

```
print("SUMMARY COMPARISON")
print("=" * 60)
print(summary_df.to_string(index=False))
print("=" * 60)
```

```
✅ Data loaded successfully. Shape: (1006, 5)
📊 Analyzing 5 assets: AAPL, AMZN, GOOGL, META, NFLX
```




```
META     0.0000
NFLX     0.0688
dtype: float64
Annual Volatility:      0.3031
Optimization Success:   True

============================================================
TASK 3: MAXIMUM SHARPE RATIO PORTFOLIO
============================================================
Optimal Weights:
AAPL     0.7874
AMZN     0.0000
GOOGL    0.1997
META     0.0000
NFLX     0.0129
dtype: float64
Expected Annual Return: 0.2801
Annual Volatility:      0.3175
Sharpe Ratio:           0.8822

============================================================
SUMMARY COMPARISON
============================================================
                      Portfolio  Exp. Return  Volatility  Sharpe Ratio
               1/n Equal-Weight     0.236048    0.326851      0.722187
Min Volatility (PyPortfolioOpt)     0.250141    0.303074      0.825348
         Min Volatility (scipy)          NaN    0.303077           NaN
               Max Sharpe Ratio     0.280111    0.317521      0.882181
============================================================
```